

Destination-based policy enforcement in Mixnets

Extending Sphinx Packets with Anonymous Credential

Aurélien Chassagne¹[0009–0008–7207–868X], Iness Ben
Guirat¹[0000–0002–8766–594X], Jan Tobias Mühlberg¹[0000–0001–5035–0576], and
Jean-Michel Dricot¹[0000–0002–8539–9940]

Université Libre de Bruxelles (ULB), Brussels, Belgium

Abstract Mixnets provide strong communication anonymity by hiding the relationship between senders and receivers. However, large-scale deployment introduces new operational challenges, as network operators may need to enforce policies on reachable destinations, for example to support service authorization, resource management, or operational agreements. Existing approaches perform destination verification at the exit gateway, where the final destination is available, but this occurs only after the traffic has already consumed the resources of the mixnet. This paper introduces an extension of the Sphinx packet format enabling privacy-preserving destination policy enforcement. Our approach integrates destination-bound credentials into the packet format, allowing a verifier to check the validity of an authority-issued credential and its binding to the encrypted final destination, without revealing the destination itself. This enables early rejection of unauthorized traffic while preserving destination privacy. Our work provides a mechanism to combine destination-based policy enforcement with the anonymity properties of mixnets, avoiding the need to reveal destinations for authorization purposes.

Keywords: Sphinx · Anonymous Credentials · Mixnet · Decentralized networks.

1 Introduction

Mixnets are a fundamental technology for privacy-preserving communication. By forwarding messages through a sequence of cryptographic relays, they make it difficult for adversaries to link senders and receivers while providing strong anonymity guarantees. The Sphinx packet format provides an efficient and secure foundation for such systems, offering compact packet processing, integrity protection, and unlinkability properties.

However, deploying mixnets at large scale raises challenges beyond cryptographic anonymity. A practical communication infrastructure may need to enforce policies related to service authorization, resource usage, or operational agreements. For example, a mixnet provider may offer subscription-based access

where users pay for the ability to anonymously communicate with a set of authorized destinations. Such a model requires a mechanism to verify that traffic is directed toward an authorized destination without revealing it.

Current mixnet architectures provide limited support for such requirements. Since intermediate mixnodes do not learn the final destination during packet processing, they cannot verify whether the destination satisfies the network’s policies. A possible solution is to perform verification at the exit gateway, once the final destination information is available. However, this postpones policy enforcement until the traffic has already traversed the mixnet and consumed bandwidth and relay resources. The key challenge is therefore to enable early rejection of unauthorized traffic without requiring intermediate mixnodes to learn the destination they are forwarding toward.

This work explores a different approach: enabling privacy-preserving policy enforcement through destination-bound credentials. We propose an extension of the Sphinx packet format that enables a verifier to validate an authority-issued credential and verify its binding to the encrypted final destination, without revealing the destination itself.

Our goal is not to introduce destination surveillance into mixnets, but to enable privacy-preserving guarantees that communications are directed toward authorized endpoints. Such a mechanism provides a middle ground between unrestricted anonymous transport and destination disclosure for policy enforcement.

2 Proposed Scheme

This section presents the proposed protocol and its integration into the Sphinx packet format. Figure 1 provides an overview of the interactions between the client, the authorities, and the mixnet.

Our protocol consists of four steps: (1) the client requests destination-specific credential shares from a set of authorities, (2) the authorities verify the request according to the network policy and issue *partial credentials*, (3) the client aggregates the partial credentials into a *destination-bound credential*, and (4) when constructing a Sphinx packet, the client rerandomizes and binds the credential to the packet header, producing a *header-bound credential* that can be verified by mixnodes without revealing the destination.

DSphinx header. The proposed scheme is based on an adaptation of the Sphinx header, called DSphinx, first introduced in [1]. The key distinction is that each element in the header is independently encoded as an elliptic curve (EC) point. Consequently, the DSphinx header is structured as a sequence of fixed-size chunks, as illustrated in Fig. ??, where each chunk corresponds to a single element represented by an EC point. Each network node along the packet route contributes for two chunks: one encoding its address and another encoding a per-hop integrity tag. Therefore, a route of length m is represented by $n = 2m + 1$ chunks, accounting for two chunks per network node and one additional chunk corresponding to the final destination. Some routing information, such as

Cite DSphinx

If places allow, out a figure specifying the header structure

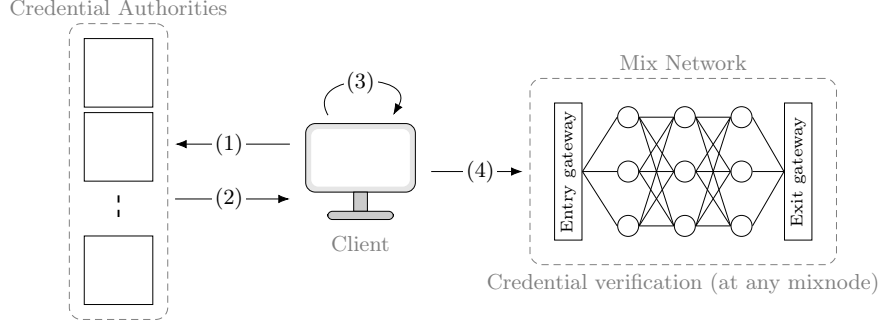


Figure 1: Overview of the credential issuance and header binding. The client obtains *partial credentials* from a set of authorities, aggregates them into a *destination-bound credential*, and finally rerandomizes and binds the credential to a Sphinx packet header, producing a *header-bound credential*.

IP addresses, must be reversibly encoded as elliptic curve points. To this end, DSphinx uses the Elligator mapping, which produces points computationally indistinguishable from randomly sampled group elements. .

2.1 Threshold Multi-Authority Setup

We consider a distributed authority composed of A independent authorities, where any subset of at least t authorities can jointly issue valid credentials. During system initialization, the authorities execute a standard distributed key generation (DKG) protocol based on verifiable secret sharing (VSS), resulting in a threshold signing key without ever reconstructing the corresponding secret.

More precisely, each authority A_k is associated with a public identifier $x_k \in \mathbb{Z}_q$ and receives a secret signing share y_k . Together, these shares define a secret polynomial f of degree $t - 1$, whose constant coefficient $y = f(0)$ is the global signing key. While no individual authority learns y , any subset of at least t authorities can jointly sign any elliptic-curve point P as $\tilde{P} = yP$ by combining their partial signature $(x_k, y_k P)$ using Lagrange interpolation at $x = 0$.

The authorities jointly authenticate the public parameters, consisting of the system public key $Y = yG'$, the public points $\tilde{N}_i = yN_i$ of the M mixnodes in the network, and the chunk generators $\tilde{G}_j = yG_j$.

$$\text{pp} = \left\{ Y, \{\tilde{N}_i\}_{i=1}^M, \{\tilde{G}_j\}_{j=1}^n \right\}. \quad (1)$$

2.2 Credential Issuance

Credential issuance is performed once for each destination and allows a client to obtain a *destination-bound* credential without revealing the destination to the authorities. The issuance process consists of two steps: partial credential issuance and its aggregation.

To update no more Elligator since incompatible with pairing curves

Update "distributed key generation" using "verifiable secret sharing" (VSS)

Partial Credential Issuance The client samples a random blinding factor $r \in \mathbb{Z}_q$ and computes the blinded representation of the destination Δ as $R = r\Delta$. The client then contacts at least t authorities and submits R together with a proof that the requested destination satisfies the applicable authorization policy.

The policy and the corresponding proof are application-dependent and are therefore outside the scope of this work. In particular, the proof may incorporate arbitrary authorization mechanisms supported by the deployment, such as a zero-knowledge proof of payment, without requiring the authorities to learn the destination itself. Our scheme only requires that the authorities can verify such a proof before issuing a credential.

Upon successful verification, each contacted authority A_k uses its secret signing share y_k to compute a *partial credential* $C_k = y_k R$, which is bound to the blinded destination R .

Credential Aggregation After collecting valid partial credentials from at least t authorities, the client combines them using Lagrange interpolation, obtaining an aggregated credential C_R bound to the blinded representation R . The client then removes the blinding factor and obtains the *destination-bound credential* $C = r^{-1}C_R = y\Delta$. Since the blinding factor r is known only by the client, the authorities learn neither Δ nor the resulting destination-bound credential during the issuance process.

2.3 Header Construction & Credential Binding

The header construction proceeds in four phases; a route and secrets derivation phase, a preprocessing phase that generates filler chunks, followed by a layer-by-layer encryption phase applied in reverse order, from the last hop to the first. Finally, the *destination-bound credential* is rerandomized and bound to the header, producing a *header-bound credential* that can be verified by any node without revealing the destination. The preprocessing and layer-by-layer encryption phases are respectively detailed in Alg. 1 and Alg. 2, and illustrated in Fig. 2 for $m = 3$. To ensure unlinkability, encryption is performed using a set of independently derived generators $\{G_j\}_{j=1}^n$, one per chunk in the header, such that no scalar relationship between any pair of generators is known.

Route and Secrets derivation. To send a message to a chosen destination Δ , for which he has the corresponding destination-bound credential $y\Delta$, the client selects a route $\{N_i\}_{i=1}^m$ and generates an initial nonce x_1 . Using this nonce and the public keys of the nodes along the route, the client derives a sequence of shared secrets $\{s_i\}_{i=1}^m$ according to Eq. (2), yielding a hop-by-hop key derivation.

$$\{s_i\}_{i=1}^m \quad \left\{ \begin{array}{l} \alpha_i = x_i G, \\ s_i = \text{Decode}(x_i K_i), \\ x_{i+1} = s_i x_i \pmod{p}, \end{array} \right. \quad (2)$$

Here, $K_i = k_i G$ is the public key of the i^{th} node in the path, p is the order of the prime subgroup, and $\text{Decode}(\cdot)$ denotes the inverse mapping from elliptic curve points to scalars. The shared secret s_i acts as a per-hop pseudorandom update that derives the next blinding factor x_{i+1} , ensuring that the cryptographic elements α_i remain unlinkable across layers.

Preprocessing Phase. The preprocessing phase prepares a vector of filler chunks ϕ according to (3). These filler chunks ensure that after each encryption layer the last two chunks evaluate to the identity element and can thus be safely truncated, ensuring a constant header size and reversibility when processed by the network nodes. The detailed procedure is specified in Alg. 1, which takes as input the set of shared secrets $\{s_i\}$ and the set of chunk generators $\{G_j\}$, and outputs $n - 1$ filler chunks, where $n = 2m + 1$ is the header size for a path of length m .

$$\phi_j = - \sum_{i=1}^m s_i G_{n+j-2i}, \quad 1 \leq j \leq n - 1. \quad (3)$$

Layered Encryption Phase. The header is initialized by concatenating the destination chunk Δ with the filler chunks ϕ , producing a n -chunks header. Encryption is then applied iteratively in reverse path order, from $i = m$ down to $i = 1$, as illustrated in Fig. ?? and detailed in Alg. 2.

1. **Chunk encryption:** Each chunk j is encrypted according Eq. (4). The last two chunks evaluate to the identity element due to the preprocessing phase and may therefore be discarded while preserving subsequent processing.

$$H_{ij} = H_{(i+1,j)} + s_i G_j \quad (4)$$

2. **Integrity computation:** Remaining chunks are used for the integrity tag:

$$\gamma_i = \text{HMAC}(\beta_i, s_i) \quad (5)$$

3. **Prepend Node:** The node address N_i and the integrity tag γ_i are prepended to H_i , producing again a n -chunks header.

Credential Binding. Before transmitting a packet, the client transforms the destination-bound credential into a *header-bound credential*. This transformation binds the credential to the current Sphinx header and pseudorandomizes its value, so that the destination-bound credential is not directly exposed in the packet.

Let H denote the header-dependent group element used to bind the credential to the current Sphinx header. It consists of two components: the public points of the mixnodes along the route, excluding the first hop, and a shared-secret-dependent contribution. For a route of length m , we define:

$$H = \underbrace{\sum_{i=2}^m N_i}_{\text{mixnode contribution}} + \underbrace{\sum_{j=1}^m s_j \left(\sum_{k=1}^{m-j+1} G_{2k-1} \right)}_{\text{shared-secret contribution}}. \quad (6)$$

Then the header-bound credential (σ) is computed by adding the destination-bound credential ($y\Delta$) to the header-dependent component H multiplied by the global secret key y . The authenticated header-dependent component yH is computed by the client using the authenticated public parameters $\{\tilde{N}_i\}_{i=2}^m$ and $\{\tilde{G}_{2*j-1}\}_{j=1}^m$, which are derived from the global secret key y .

$$\sigma = y(\Delta + H) = y\Delta + yH, \quad (7)$$

The resulting credential is both bound to the current Sphinx header and pseudorandomized by the header-dependent components. In particular, two packets carrying credentials for the same destination result in different header-bound credentials since their header-dependent values differ.

Algorithm 1 Preprocessing

Input:

$[s_i]_{i=1..m}$ shared secrets
 $[G_j]_{j=1..n}$ chunk generators

Output:

ϕ filler chunks

```

1:  $m \leftarrow \text{pathLength}$ 
2:  $n \leftarrow 2m + 1$ 
3:  $\phi \leftarrow$  list of  $(n - 1)$  identity points
4: for  $i = m$  down to 1 do
5:   for  $j = 2(m - i + 1)$  to  $n$  do
6:      $l \leftarrow 2i + j - n$ 
7:      $\phi[l] \leftarrow \phi[l] - s_i G_j$ 
8:   end for
9: end for
10: return  $\phi$ 

```

Algorithm 2 Encryption

Input:

Δ Destination
 $[N_i]_{i=1..m}$ Route

Output:

H header chunks

```

1:  $H \leftarrow [\Delta \parallel \phi]$ 
2: for  $i = m$  down to 1 do
3:   for  $j = 1$  to  $n$  do
4:      $H[j] \leftarrow H[j] + s_i G_j$ 
5:   end for
6:    $H \leftarrow H[1 : n - 2]$ 
7:    $\gamma_i \leftarrow \text{HMAC}(H, s_i)$ 
8:    $H \leftarrow [N_i \parallel \gamma_i \parallel H]$ 
9: end for
10: return  $H$ 

```

2.4 Packet Processing & Credential Verification

Upon receiving a packet, mixnode i extracts the cryptographic element α_i , the integrity tag γ_i , and the encrypted routing information β_i . As in the original Sphinx design, the node checks whether α_i has been previously observed since its last key rotation and discards duplicates to prevent replay attacks. The header is processed as illustrated in Fig. 3.

Credential Verification. First the mixnode verifies the header-bound credential σ using the system public key $Y = yG'$. The verification considers all header chunks that may contain the encrypted final destination (i.e. all odd-indexed β

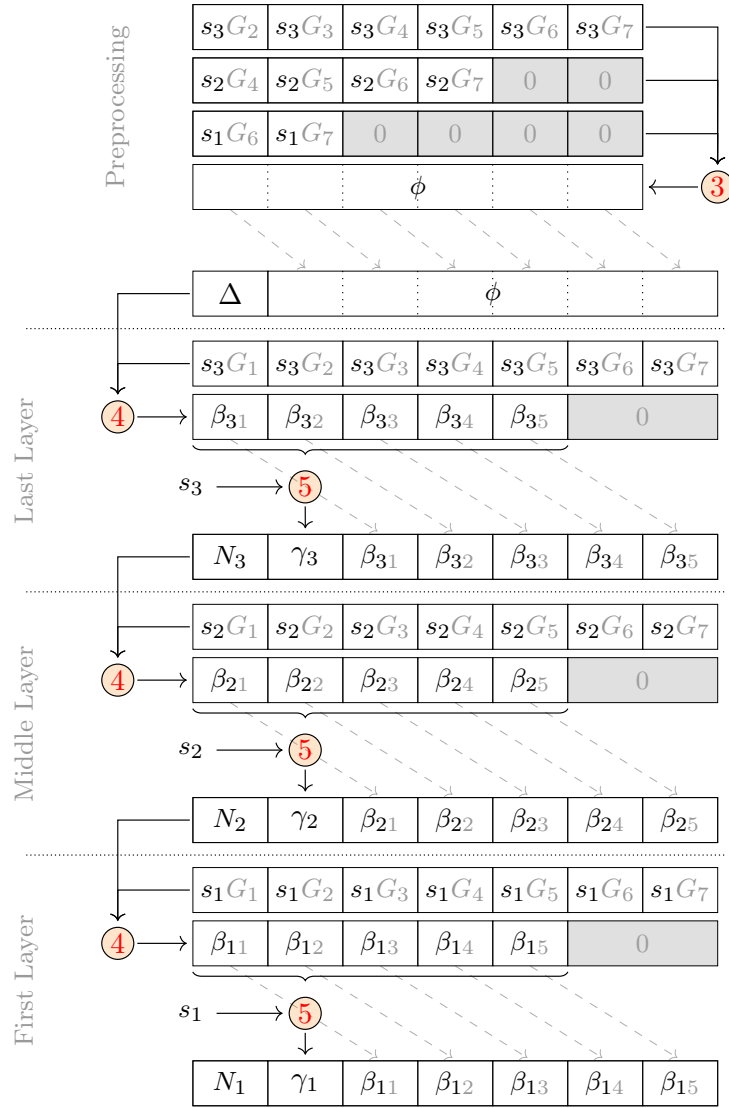


Figure 2: DSphinx header construction for a path of length $m = 3$ (without credential binding). Circled numbers $\textcircled{x}$ refers to Equation x in the paper.

chunks), and therefore does not require the mixnode to determine which chunk actually contains the destination. The credential is accepted if the following pairing equation holds; otherwise, the packet is discarded:

$$e(\sigma, G') = e\left(\sum_{i=1}^m \beta_{2i-1}, Y\right), \quad (8)$$

Since all candidate destination chunks are included in the verification, a successful verification establishes that σ is a valid authority-issued credential bound to the encrypted destination carried by the header, without revealing which candidate chunk contains that destination.

Integrity Check. The mixnode derives the shared secret s_i by multiplying the cryptographic element α_i with its private key k_i , and then hashing the resulting point into a scalar as in (9). The shared secret s_i is then used to verify the integrity tag in (10).

$$s_i = \text{HashToScalar}(k_i \alpha_i) \quad (9)$$

$$\gamma_i \stackrel{?}{=} \text{HMAC}(\beta_i, s_i) \quad (10)$$

$$(11)$$

Decryption. If verification succeeds, the node appends two identity chunks and decrypts the header as in (12). The first decrypted chunk β'_{i1} reveals the next hop N_{i+1} . The second chunk β'_{i2} contains the integrity tag γ_{i+1} for the next hop, while the remaining chunks form the next encrypted routing information β_{i+1} .

$$\beta'_{ij} = \beta_{ij} - s_i G_j \quad (12)$$

Update. Finally, the node updates the cryptographic element according to (13). It also updates the credential according to (14) using the authenticated next hop public point $\tilde{N}_{i+1}$. Now the packet has been processed by mixnode i , it can be shuffled, delayed and forwarded to the next hop.

$$\alpha_{i+1} = s_i \alpha_i \quad (13)$$

$$\sigma_{i+1} = \sigma_i - s_i \left(\sum_{k=1}^{m+1} \tilde{G}_{2k-1} \right) - \tilde{N}_{i+1} \quad (14)$$

3 Performance evaluation

We evaluate the computational overhead introduced by DSphinx and by the destination-bound credential mechanism, as well as their scalability with respect to the main system parameters. We implemented a Python prototype¹ to

¹ <https://github.com/AurelienCha/Mixnet-Credential>

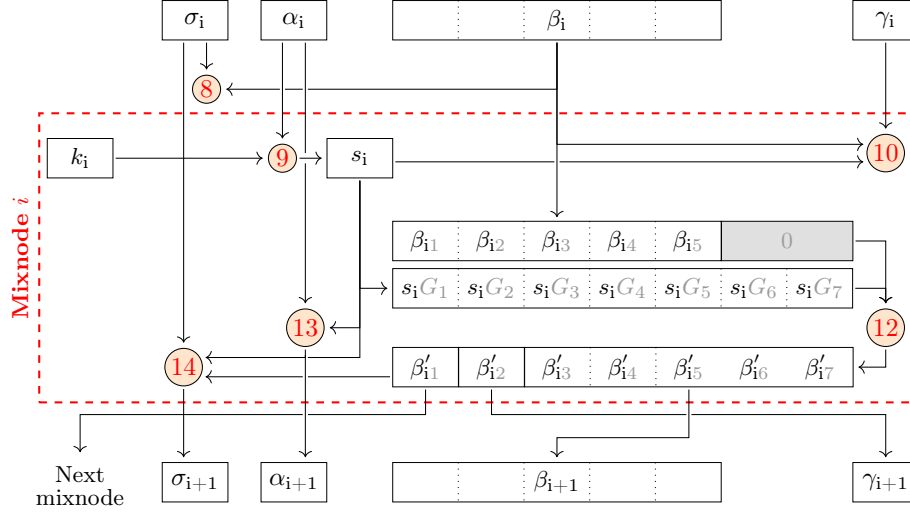


Figure 3: DSphinx header of path length $m = 3$ and its credential processed by mixnode i . Circled numbers $\otimes$ refers to Equation x in the paper.

compare three configurations: DSphinx with credentials, vanilla DSphinx, and the original Sphinx packet construction of Danezis, which we use as a baseline.

All measurements reported in this section were obtained on a 13th Gen Intel Core i7-13700HX CPU running at 3.70 GHz. Unless otherwise stated, the benchmark simulates a network of 50 clients, 50 mixnodes, and 50 credential authorities, with each client sending five messages to itself.

3.1 Parameter sensitivity

We first perform a preliminary sensitivity analysis to identify the parameters that most strongly affect the computational cost of each protocol phase. For each phase and parameter x , we fit the empirical power-law model $T(x) \propto x^\lambda$ using a linear regression in log-log space. The exponent λ characterizes the observed scaling, with values close to 0, 1, and 2 indicating approximately constant, linear, and quadratic behavior, respectively. The coefficient of determination R^2 indicates the goodness of fit.

As shown in Figure 4, the clearest dependencies are observed for the path length m and reconstruction threshold k . Header Construction exhibits near-quadratic scaling with m ($\lambda = 2.03$, $R^2 = 0.88$), while Setup and Credential Issuance exhibit approximately linear-to-superlinear scaling with the threshold ($\lambda = 1.32$, $R^2 = 0.86$ and $\lambda = 1.03$, $R^2 = 0.71$, respectively). The remaining parameters show weak fits and provide little evidence of consistent scaling. We therefore focus the remainder of the evaluation on the path length m and reconstruction threshold k .

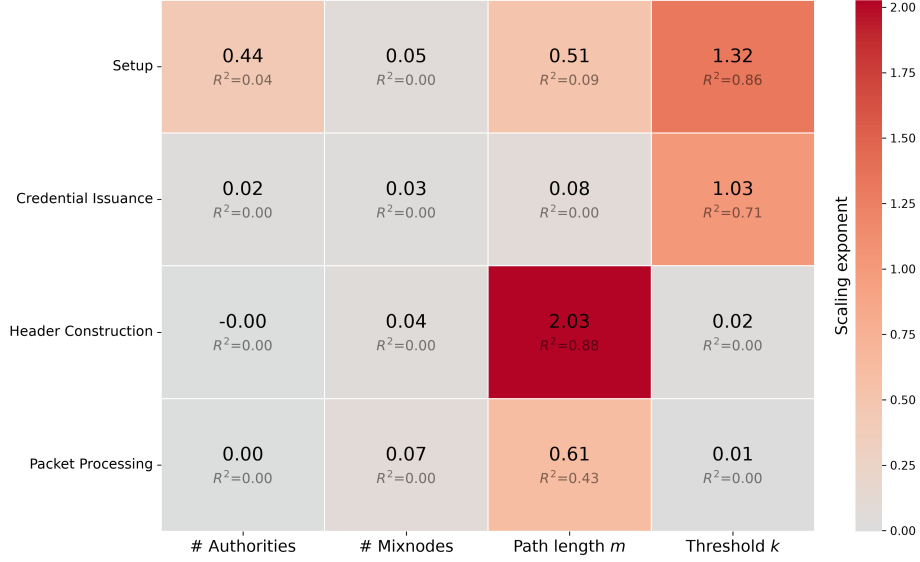


Figure 4: Empirical scaling of the protocol phases with respect to the main system parameters. Each cell shows the scaling exponent λ , with its goodness of fit R^2 .

3.2 Packet construction and processing

For the first experiment, we compare the computational cost of packet’s header generation at the client and packet processing at a mixnode for DSphinx with credential, vanilla DSphinx, and the Sphinx baseline. We vary the path length m from 3 to 7 hops with a fixed reconstruction threshold $k = 20$. This range follows the maximum path length supported by the original Sphinx implementation used as our baseline. The results are shown in Figure 5.

From the client perspective, DSphinx requires approximately two to three times more computation than the Sphinx baseline for header generation. Nevertheless, the absolute cost remains below 12 ms for a seven-hop path. Adding the credential introduces only a small additional cost, primarily associated with the credential-binding operation of Equation (7).

At the mixnode, vanilla DSphinx introduces an additional cost of approximately 1 ms compared with Sphinx, while credential verification adds approximately 2 ms. Although this corresponds to an absolute overhead of less than 3 ms, the relative overhead is substantial because baseline Sphinx packet processing remains approximately constant at 0.2 ms in our experiment. Therefore the credential version requires approximately 10-15 times more computation than the Sphinx baseline. The dominant source of this additional cost is the two pairing operations required for credential verification in Equation (8).

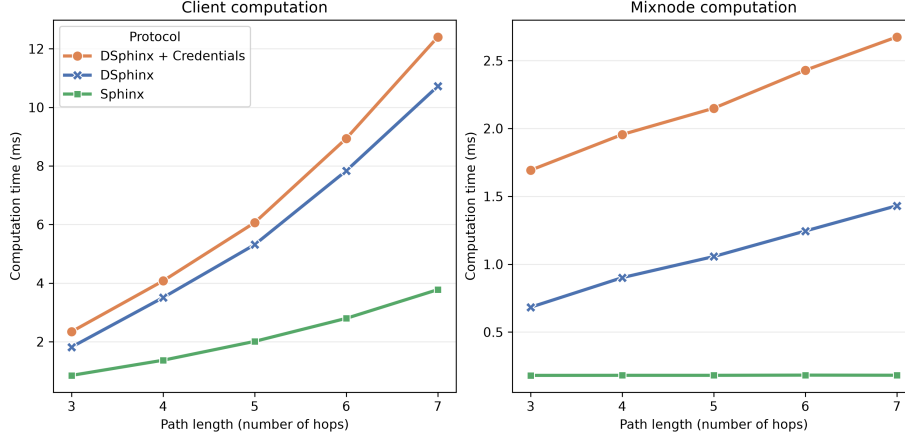


Figure 5: Computation time for packet’s header generation at the client and packet processing at a mixnode as a function of the path length m , ranging from 3 to 7 hops.

3.3 Setup and credential issuance

We next evaluate the additional procedures required by our scheme, the system setup and credential issuance. The setup is performed once during system initialization, whereas credential issuance is performed by a client for each destination for which it requires a destination-bound credential. As indicated by Figure 4, the reconstruction threshold k is the dominant parameter for these operations. We therefore vary k while fixing the path length to $m = 5$. Figure 6 presents the corresponding execution times.

The setup procedure is the most computationally expensive operation in our scheme. For $k = 50$, it requires up to approximately 500 ms in our prototype. This cost is incurred only once during initialization. The measured cost is primarily attributed to the Lagrange-polynomial interpolation of degree $k - 1$ used to authenticate the public parameters.

Credential issuance requires approximately 10-70 ms at the client, including the request and the aggregation of the partial credentials returned by the authorities. In this experiment, the computational cost at an authority is negligible because we consider only the basic credential-issuance operation and do not include the verification of the authorization proof that would be required in a complete deployment. In the current prototype, an authority therefore performs a single elliptic-curve multiplication. These results indicate that the main additional cost introduced by the credential mechanism occurs during setup and at mixnodes during credential verification, whereas the client-side cost of credential issuance remains in the millisecond range.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

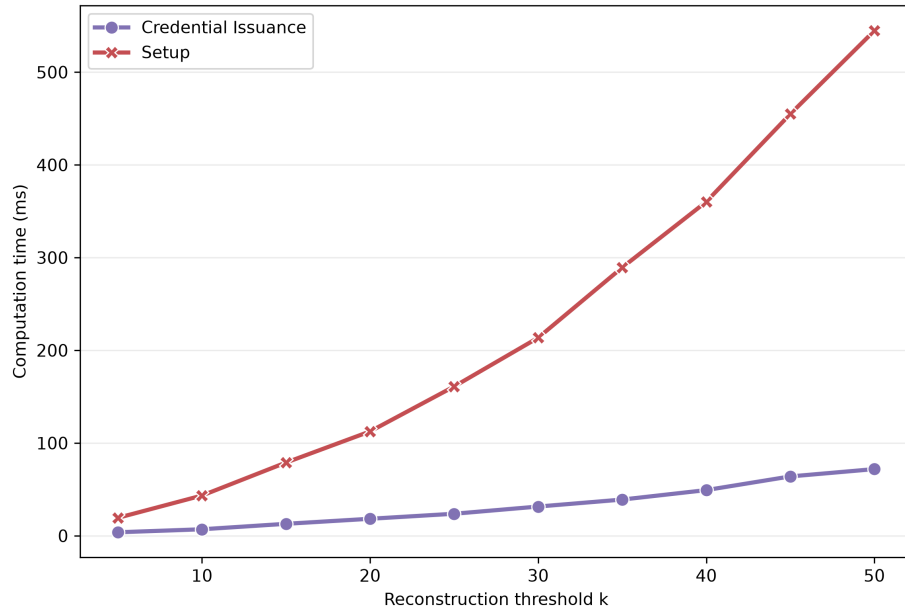


Figure 6: Computation time introduced by the setup procedure and by credential issuance as a function of the reconstruction threshold k . Setup is performed once, whereas credential issuance is performed by a client to obtain a destination-bound credential.

References